

Aperture — Low-Level Design (LLD Index)

Version: v1.0

Document Type: Low-Level Design (Index)

Related Documents

- Product Requirements Document
 - Development Roadmap
 - System Architecture
 - Technical Architecture
-

1. Purpose

This document defines the implementation philosophy for Aperture.

Unlike the System Architecture document, which explains how the platform is organized, the Low-Level Design (LLD) explains how each subsystem should be implemented.

To improve maintainability, Aperture's implementation details are split into independent module documents rather than one monolithic specification.

Each module follows the same engineering conventions described in this document.

2. LLD Philosophy

Every module should be:

- Self-contained
- Independently testable
- Clearly documented
- Easy to understand
- Easy to evolve

Business logic should be isolated from infrastructure concerns.

Implementation should prioritize readability and maintainability over cleverness.

3. Module Structure

Every domain follows the same high-level structure.

```
Module
|
├─ API Layer
├─ Service Layer
├─ Repository Layer
├─ Domain Models
├─ Database Models
├─ Validation
├─ Background Tasks
├─ Tests
└─ Documentation
```

Responsibilities should not overlap across layers.

4. Standard Layers

API Layer

Responsible for:

- HTTP endpoints
- request validation
- response serialization
- authentication hooks
- authorization checks

Contains no business logic.

Service Layer

Responsible for:

- business rules
- workflows
- orchestration
- transactions
- cross-module coordination

This is the primary location for domain logic.

Repository Layer

Responsible only for data access.

Responsibilities include:

- querying
- persistence
- pagination
- filtering

Repositories should not contain business decisions.

Domain Models

Represent business concepts.

Examples:

- User
- Movie
- Review
- Watchlist

Domain models should remain independent of transport and persistence concerns wherever practical.

Validation

Validation occurs as early as possible.

Types include:

- request validation
- business validation
- authorization validation
- database constraint validation

Never rely solely on frontend validation.

5. Module Documentation Template

Every module-specific LLD should contain:

1. Overview
2. Responsibilities
3. Dependencies
4. Public APIs
5. Internal Services
6. Database Models
7. Validation Rules
8. Background Jobs
9. Error Handling
10. Security Considerations
11. Performance Considerations
12. Testing Strategy
13. Future Evolution

This creates consistency across the entire codebase.

6. Cross-Module Communication

Modules communicate through well-defined service interfaces.

Avoid exposing repositories directly across domains.

Example:

Authentication → Users

Users → Reviews

Reviews → Discovery

Each dependency should be intentional and documented.

7. Error Handling

Errors should be categorized consistently.

Examples:

- Validation Errors
- Authentication Errors
- Authorization Errors
- Business Rule Violations
- Resource Not Found
- External Service Failures
- Internal Server Errors

Responses should be predictable and machine-readable.

8. Logging

Every module should emit structured logs for meaningful events.

Examples:

- Login success
- Review creation
- Metadata synchronization
- Recommendation generation

Logs should support debugging and monitoring without exposing sensitive information.

9. Testing Expectations

Every module should include:

- Unit tests for business logic
- Integration tests for persistence
- API tests for endpoints
- Edge case coverage
- Failure scenario testing

Testing is considered part of implementation, not a separate phase.

10. Module Dependency Principles

Dependencies should point inward toward business logic.

Avoid circular dependencies.

If two modules become tightly coupled, reconsider the domain boundaries rather than introducing shortcuts.

11. Initial Module Breakdown

The first version of Aperture consists of the following implementation modules:

1. Authentication
2. Users
3. Metadata
4. Search
5. Reviews
6. Lists
7. Discovery
8. Streaming
9. Notifications
10. Administration

Future modules include:

- Recommendation Engine
 - AI Platform
 - Movie Spaces
 - Analytics
 - Moderation
 - Reporting
-

12. Recommended Implementation Order

Development should generally follow this sequence:

1. Authentication
2. Users
3. Metadata
4. Search
5. Lists
6. Reviews
7. Discovery
8. Streaming
9. Notifications
10. Administration

This order minimizes blocking dependencies and aligns with the Development Roadmap.

13. Relationship to the Codebase

Each module document maps directly to a corresponding package in the backend.

As the project evolves, changes to implementation should be reflected in the relevant module LLD rather than duplicated across multiple documents.

The LLD should remain a practical implementation guide rather than an exhaustive code reference.

14. Summary

The Low-Level Design documentation provides implementation guidance while preserving modularity.

Every subsystem is documented independently, allowing developers to understand, build, test, and evolve one part of the platform without needing to understand the entire system in detail.

This approach keeps the documentation scalable in the same way the software architecture is designed to scale.